LAB 6 : Heap

[CO2]

Instructions for students:

- Complete the following methods.
- You may use Java / Python to complete the tasks.
- DO NOT CREATE a separate folder for each task.

NOTE:

- **YOU CANNOT USE ANY OTHER DATA STRUCTURE OTHER THAN ARRAY.**
- **YOUR CODE SHOULD WORK FOR ANY VALID INPUTS.**

Python List, Negative indexing and append() is STRICTLY prohibited

Lab Tasks: 3 tasks (1~3)
Assignment Tasks: 3 tasks (4~6)

<u>Total Assignment Mark: 3*5=15</u>

You were given instructions and driver code for the last 6 lab files. However, for this lab, only Task#1 driver codes will be given. Write the other driver codes yourself in your preferred language (Java / Python).

LAB TASKS

Task1: Implementing an Array-Based Min Heap

Driver Codes for Task#1 only [Java Task#1 Template] & [Python Task#1 Template]

A **Min Heap** is a specialized binary tree where the value of each node is **less than or equal to its children**. This ensures that the **smallest element is always at the root**.

In this implementation, the heap is represented using an **array**, where the tree structure is maintained through 1-based index relationships.

The MinHeap class contains the following private instance variables:

An Integer array **heap**	stores the elements of the heap
An integer **size**	represents the current number of elements in the heap
An integer **capacity**	represents the maximum number of elements the heap can hold

Instance Methods to Implement	Steps
insert(value) Description: Inserts a new value into the heap while maintaining the heap property.	1. Check if the heap is full (if **size** is greater or equal to the **capacity**) → If full, print "Heap is full" and do not insert 2. Insert the value at the end of the array 3. Restore heap property using **heapifyUp**
extractMin() Description: Removes and returns the smallest element (root of the heap).	1. If the heap is empty, return null 2. Store the root value ***heap[1]*** (this will be returned) 3. Move the last element to the root position 4. Decrease the size of the heap 5. Restore heap property using **heapifyDown** 6. Return the stored minimum value
peek() Description: Returns the smallest element without removing it.	1. If the heap array is empty, return ***null*** 2. Return the root element ***(heap[1])***
heapifyUp(index) Description: A private method that restores the heap property by moving an element upward. **When used:** After insertion	1. Compare the current element with its parent 2. If the parent is greater, swap them 3. Continue until the heap property is satisfied or root is reached

heapifyDown(index) Description: A private method that restores the heap property by moving an element downward. **When used:** After extracting the minimum element	1. Compare the current element with its children 2. Swap with the smallest child if needed 3. Continue until the heap property is satisfied or reached leaf
isEmpty()	Return **True** if the heap size is 0, otherwise return **False**

Task 2:

Complete task 1 for MaxHeap.

Note:
For MaxHeap class, the instance variables & instance methods would be the same as MinHeap except for extractMin(), instead it'll be named extractMax().

The MaxHeap sorts the element in ascending order.

The instance variables must be Private.

Task 3:

Add a ***static method*** to the class from **Task#1** or **Task#2** (depends on whether you want to use MaxHeap or MinHeap) that performs **out-of-place heap sort** using a heap (min-heap or max-heap). The method will be called ***outPlaceHeapsort()*** and it takes an array in its parameter. The method will do the following:

- Create a Min-Heap or Max-Heap instance.
- Loop through the given array:
    - Insert each element into the heap.
- Loop until the Max-Heap or Min-Heap is empty:
    - Extract the minimum/maximum element from the minheap/maxheap.
    - put them in their sorted order in the original array

Note: The time complexity of the solution is going to be `n*log(n)`.

You can utilize the Classes you created for the Lab Task by creating their objects to complete the Assignment Tasks. Just like Stack & Queue tasks, you'll create MaxHeap/MinHeap objects and call their methods. You cannot access the instance variables from outside since they're private.

ASSIGNMENT TASKS

Task 4 [5 marks] :

You are given an array of n tasks, where the value of index i represents the processing time for i-th task. You are also given m machines, each capable of processing one task at a time. Your goal is to distribute the tasks among the machines such that the overall completion time for each machine is minimized. Represent the output as an array where each index corresponds to a machine, and the value at each index is the total processing time of tasks assigned to that machine.

Input Format:

- **tasks:** An array of integers where each integer represents the processing time of a task.
- **m:** An integer representing the number of machines.

Output Format:

- An array of integers of size m, where the value at index i represents the total processing time of tasks assigned to the i-th machine.

Sample Given Input:
tasks = [2, 4, 7, 1, 6]
m = 4

Sample Output:
[2, 4, 7, 7]

Explanation:
To simplify the use of a heap, we focus only on the current load of each machine (not its index). The min-heap will store the current loads of the machines. Initially, all machines will have a load of 0.
Once all tasks are assigned, the heap will contain the loads of all machines. The array of this heap will represent the output.

Steps to Solve:

- ➔ Initialize a min-heap object with m length, all set to 0 (representing the load of each machine).
- ➔ For each task, do the following:
  - ◆ Extract the smallest load from the heap.
  - ◆ Add the task's processing time to this load.
  - ◆ Reinsert the updated load into the heap.
- ➔ The final array representation for your heap is the answer.

Example Walkthrough:

Initial Heap:

Heap Array	[0, 0, 0, 0]
Heap Visual	0 / \ 0 0 / 0

Task 1 (Processing Time = 2): Assign to the machine with load 0 (current minimum load).

	Extract the smallest	Add the extracted with current & reinsert
Heap Array	0 extracted After extraction: [0, 0, 0, None/null]	0+2=2 is inserted After insertion: [0, 0, 0, 2]
Heap Visual	0 / \ 0 0	0 / \ 0 0 / 2

Task 2 (Processing Time = 4): Assign to the machine with load 0 (current minimum load).

	Extract the smallest			Add the extracted with current & reinsert
Heap Array	0 extracted After extraction: [0, 0, 2, None/null]			0+4=4 is inserted After insertion: [0, 0, 2, 4]
Heap Visual	0 / \ 0 0 / 2	2 / \ 0 0	0 / \ 0 2	0 / \ 0 2 / 4

Task 3 (Processing Time = 7): Assign to the machine with load 0 (current minimum load)

	Extract the smallest			Add the extracted with current & reinsert
Heap Array	0 extracted After extraction: [0, 4, 2, None/null]			0+7=7 is inserted After insertion: [0, 4, 2, 7]
Heap Visual	0 / \ 0 2 / 4	4 / \ 0 2	0 / \ 4 2	0 / \ 4 2 / 7

Task 4 (Processing Time = 1): Assign to the machine with load 0 (current minimum load).

	Extract the smallest			Add the extracted with current & reinsert		
Heap Array	0 extracted After extraction: [0, 4, 2, None/null]			0+1=1 is inserted After insertion: [1, 2, 7, 4]		
Heap Visual	0 / \ 4 2 / 7	7 / \ 4 2	2 / \ 4 7	2 / \ 4 7 / 1	2 / \ 1 7 / 4	1 / \ 2 7 / 4

Task 5 (Processing Time = 6): Assign to the machine with the smallest load, that is 1, and add the current processing time (6) with this.

	Extract the smallest			Add the extracted with current & reinsert
Heap Array	1 extracted After extraction: [2, 4, 7, None/null]			1+6=7 is inserted After insertion: [2, 4, 7, 7]
Heap Visual	1 / \ 2 7 / 4	4 / \ 2 7	2 / \ 4 7	2 / \ 4 7 / 7

Final Output:
Finally, the array representation of the heap is [2, 4, 7, 7]

Hints:

1. Tasks must be assigned one at a time to the machine with the current smallest load.
2. Use a heap (you have built before) to efficiently find the machine with the smallest load.
3. You can create a function that will receive tasks and m and return the final heap for simplicity

<u>Task 5 [5 marks]</u> :

You are given an array of integers and a number k. Your task is to find the top k largest elements from the list. Use a max-heap to solve this problem efficiently.

Input Format:

- nums: An array of integers.
- k: An integer representing the number of largest elements to find.

Output Format:

- An array of k integers representing the largest elements in descending order.

Sample Input:
nums = [4, 10, 2, 8, 6, 7]
k = 3
Output:
[10, 8, 7]

Explanation:

To solve this, we use a max-heap because it allows efficient retrieval of the largest elements. The process involves:

1. Insert all numbers from the input list into a max-heap.
2. Create a new array with size k.
3. Extract the maximum k times and store it to the new array to get the top k largest elements.
4. Return the result array in descending order.

Steps to Solve:

1. Build a max-heap where the length of the array will be equal to length of nums.
2. Extract the maximum k times and store the values in the newly created result array.
3. Return the result array.

Hints:

1. You must use a max-heap **(created above in Task2)** to solve the problem.
2. k is always smaller than or equal to the size of the array.
3. You can create a function that will receive the nums array and k for simplicity.

For TASK#6 You need build a new custom MaxHeap

Task 6 [5 marks] : Task Scheduler with Priority Queue

You are given two arrays: one containing task names and another containing their corresponding priorities. Your task is to process all tasks in order of their priority using a Max-heap. Tasks with higher priority values should be processed first.

Input Format:

- *`task_names:`* An array of strings where each element represents a task name.
- *`priorities:`* An array of integers where each element represents the priority of the corresponding task.

Output Format:

- An array of strings representing the task names in the order they should be processed (highest priority first).

Steps to Solve:

1. Build a customized max-heap where each element contains both the task name and its priority.
2. Iterate through both arrays simultaneously and insert each (task_name, priority) pair into the max-heap (heap should compare based on priority).
3. Extract the maximum repeatedly until the heap is empty, storing each task name in the result array.
4. Return the result array containing all task names in priority order.

Sample Input
task_names = ["Email", "Meeting", "Code Review", "Lunch", "Debug"] priorities = [2, 5, 3, 1, 4]
Sample Returned Priorities Array
["Meeting", "Debug", "Code Review", "Email", "Lunch"]
Explanation
To solve this problem, we use a max-heap like a priority queue because it allows efficient retrieval of the highest priority task. The process involves: 1. Combine task names with their priorities and insert all tasks into a max-heap based on their priority values. 2. Create a result array to store task names in processing order. 3. Extract the maximum (highest priority task) repeatedly until the heap is empty. 4. Return the result array containing task names in descending order of priority.